Firebase: подключаем к Flutter-проекту

Этот гайд проведёт вас по всем шагам: от создания проекта на сайте Firebase до первого пользователя в вашем приложении. Дальше — простой экран входа, создание базы Firestore, публикация веб-версии на Firebase Hosting, что коммитить в Git, таблица частых ошибок и шпаргалка.

Работа идёт в двух местах. Первое — **консоль Firebase**, сайт console.firebase.google.com. Там вы создаёте проект и включаете нужные сервисы. Второе — **терминал**. Команды в тексте выделены отдельным шрифтом, например: **flutter --version**. После ввода команды нажмите Enter.

Важно: в учебном проекте используйте только тестовые данные. Регистрируйте пользователей с почтами вида **test1@test.com**, не вводите свои настоящие данные и данные друзей. Почему так: Firebase хранит данные на серверах за пределами России, а закон требует, чтобы персональные данные россиян сначала записывались в базы на территории России. Для учёбы это не проблема, для реального продукта — важно.

1. Что понадобится

Проверьте всё по списку до начала работы. Каждый пункт нужен один раз.

- **Google-аккаунт.** Через него вы входите в консоль Firebase. Подойдёт обычная почта Gmail.
- **Node.js версии 22 или новее.** Это программа, через которую устанавливается Firebase CLI. Проверьте версию командой **node -v**. Если Node нет или версия ниже 22 — скачайте версию LTS с сайта nodejs.org и установите с настройками по умолчанию.
- **Flutter-проект, который запускается.** Ваш каталог из домашних заданий. Перед началом убедитесь, что **flutter run** работает без ошибок.

Где открыть терминал: в VS Code — меню Terminal, пункт New Terminal; в Android Studio — вкладка Terminal внизу окна. Терминал внутри редактора сразу открывается в папке проекта, это удобно.

2. Создаём проект в консоли Firebase

Проект Firebase — это «контейнер» на стороне Google. В нём живут ваши пользователи, база данных и настройки. Одному Flutter-приложению соответствует один проект Firebase.

1. Откройте сайт console.firebase.google.com и войдите своим Google-аккаунтом. При первом входе согласитесь с условиями использования.
2. Нажмите Create a project (или «Создать проект», если консоль на русском).
3. Введите название проекта латиницей, например **catalog-ivanov**. Под названием Firebase покажет ID проекта — уникальное имя, по которому его потом найдут CLI-инструменты. Запомните его.
4. Если консоль предложит включить Gemini in Firebase (ИИ-помощника), можно отказаться — для курса он не нужен.
5. На шаге Google Analytics выключите переключатель «Enable Google Analytics for this project». Так меньше шагов и меньше собираемых данных.
6. Нажмите Create project и подождите около минуты. Когда появится «Your new project is ready», нажмите Continue.

Вы попадёте на главную страницу проекта (Project Overview). Слева внизу виден тариф: Spark — бесплатный. Ничего не меняйте и не нажимайте Upgrade: банковская карта для курса не нужна.

Приложения Android, iOS и Web на главной странице вручную добавлять не нужно. Это за вас сделает FlutterFire CLI в разделе 5.

3. Устанавливаем Firebase CLI

Firebase CLI — программа для терминала, которая умеет входить в ваш Google-аккаунт, видеть ваши проекты и публиковать сайты. Ставится один раз на компьютер.

```
npm install -g firebase-tools
```

Если команда завершилась без красных ошибок, проверьте установку:

```
firebase --version
```

В ответ должен прийти номер версии, например **15.x.x**. Если что-то пошло не так, посмотрите варианты ниже.

- macOS: ошибка EACCES (permission denied). Повторите команду с **sudo** в начале и введите пароль от компьютера: **sudo npm install -g firebase-tools**.
- Windows: «выполнение сценариев отключено в этой системе». Это ограничение PowerShell. Выполните один раз **Set-ExecutionPolicy -Scope CurrentUser RemoteSigned**, ответьте «Y» и повторите команду. Другой вариант — использовать терминал cmd вместо PowerShell.
- Много жёлтых предупреждений `npm WARN EBADENGINE`. У вас старый Node: части пакетов нужен Node 22 или новее. Обновите Node (раздел 1) и повторите установку.
- Нет Node.js и не хочется ставить (macOS, Linux). Есть установщик без Node: **curl -sL https://firebase.tools | bash**.

Входим в аккаунт

```
firebase login
```

Терминал может спросить, разрешить ли собирать статистику использования CLI. Ответ ни на что не влияет: можно ввести **n** и нажать Enter. Затем откроется браузер: выберите тот же Google-аккаунт, в котором

создали проект, и нажмите «Разрешить». В терминале появится «Success! Logged in as ...».

Проверим, что CLI видит ваш проект:

```
firebase projects:list
```

В таблице должен быть ваш **catalog-ivanov**. Если его нет — CLI вошёл не тем Google-аккаунтом, которым вы создавали проект в браузере. Проверьте почту в правом верхнем углу консоли Firebase и добавьте этот аккаунт в CLI:

```
firebase login:add
firebase login:use ваша.почта@gmail.com
firebase projects:list
```

login:add добавляет второй аккаунт, не удаляя первый, а **login:use** делает его активным.

4. Устанавливаем FlutterFire CLI

FlutterFire CLI — надстройка над Firebase CLI специально для Flutter. Она настраивает ваш Flutter-проект: регистрирует платформы в консоли, раскладывает конфигурационные файлы и создаёт файл с ключами. Без неё всё это пришлось бы делать руками для каждой платформы отдельно.

```
dart pub global activate flutterfire_cli
```

Проверка:

```
flutterfire --version
```

Если пишет «flutterfire: command not found»

Это самая частая ошибка. Программа установилась, но папка, куда Dart кладёт такие утилиты, не прописана в PATH — списке мест, где терминал ищет команды. Обычно Dart сам пишет об этом жёлтым предупреждением после установки.

- macOS (терминал zsh). Выполните **echo 'export PATH="$PATH:$HOME/.pub-cache/bin"' >> ~/.zshrc**, затем закройте терминал и откройте заново.
- Windows. Откройте «Пуск» → «Изменение системных переменных среды» → «Переменные среды». В переменную Path пользователя добавьте строку **%LOCALAPPDATA%\Pub\Cache\bin**. Перезапустите терминал и редактор.

После этого **flutterfire --version** должна ответить номером версии.

5. Настраиваем проект: flutterfire configure

Откройте терминал в корне своего Flutter-проекта — в той папке, где лежит файл **pubspec.yaml**. Проверить можно командой **ls** (на Windows **dir**): в списке должен быть **pubspec.yaml**.

Подставьте ID своего проекта. Его видно в консоли Firebase: Project settings → General → Project ID. Консоль сама подсказывает эту команду, если на главной странице проекта нажать значок Flutter.

```
flutterfire configure --project=catalog-ivanov --platforms=android,web
```

Флаг **--platforms=android,web** сразу выбирает платформы: Android для телефона и эмулятора, web для запуска в Chrome. iOS без дополнительной настройки на Mac часто падает (см. ниже), поэтому для курса берём только Android и web.

Если команда спросит про Android application id или обновление файлов Gradle, соглашайтесь с вариантом по умолчанию: просто Enter или **y**.

В конце появится сообщение «Firebase configuration file lib/firebase_options.dart generated successfully». В консоли Firebase на главной странице теперь видны ваши приложения: Android, Web и, возможно, iOS.

Таблица 1. Что создаёт flutterfire configure

Файл	**Что внутри**
lib/firebase_options.dart	Ключи проекта для всех выбранных платформ. Этот файл вы подключаете в коде.
android/app/google-services.json	Конфигурация для Android.
ios/Runner/GoogleService-Info.plist	Конфигурация для iOS (если выбрали ios).
firebase.json	Настройки FlutterFire в корне проекта.
Изменения в Gradle	В файлы android/... добавляется плагин google-services.

Источник: составлено автором.

Если нужен iOS (только macOS)

Если при выборе **ios** команда падает с длинной ошибкой про **XCLocalSwiftPackageReference**, значит, у встроенного в macOS Ruby слишком старая библиотека **xcodeproj**. Обновлять системный Ruby не стоит — поставьте отдельный через Homebrew:

```
brew install ruby
echo 'export PATH="/opt/homebrew/opt/ruby/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
which ruby      # должно быть /opt/homebrew/opt/ruby/bin/ruby
gem install xcodeproj
flutterfire configure --project=catalog-ivanov
```

На Intel-маках путь начинается с **/usr/local/opt/ruby/bin**. Если **brew install ruby** начинает собирать llvm или rust из исходников (это часы) — отмените через Ctrl+C и оставайтесь на Android и web.

Когда запускать команду ещё раз: если добавили новую платформу или подключаете некоторые сервисы Firebase (например, Crashlytics). Повторный запуск безопасен — он просто обновит файлы.

6. Подключаем пакеты и запускаем Firebase

Пакеты

firebase_core — основной пакет, без него не работает ни один сервис Firebase. **firebase_auth** — пакет для входа пользователей.

```
flutter pub add firebase_core firebase_auth
```

Код в main.dart

Firebase нужно запустить один раз, до того как приложение начнёт работать. Измените функцию **main** так:

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

void main() async {
 WidgetsFlutterBinding.ensureInitialized();
 await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
 );
 runApp(const MyApp());
}
```

Что здесь происходит:

- **main** стала **async**, потому что **Firebase.initializeApp** возвращает Future — подключение занимает время, и мы ждём его через **await**. Это то же самое, что мы делали с запросами к API.
- **WidgetsFlutterBinding.ensureInitialized()** готовит Flutter к работе с платформой (Android, iOS, браузер) ещё до **runApp**. Без этой строки Firebase не сможет запуститься.

- **DefaultFirebaseOptions.currentPlatform** берёт ключи для той платформы, на которой сейчас запущено приложение, из файла **firebase_options.dart**.

Запустите приложение. Если оно открылось как обычно и в консоли отладки нет красных ошибок — Firebase подключён.

7. Включаем вход по email и паролю

Сервисы Firebase по умолчанию выключены. Вход нужно включить в консоли.

1. В консоли Firebase слева откройте раздел **Build** (или «Сборка»), затем **Authentication**.
2. Нажмите **Get started**.
3. Перейдите на вкладку **Sign-in method**. В списке **Native providers** выберите **Email/Password**.
4. Включите первый переключатель **Email/Password**. Второй, «Email link (passwordless sign-in)», не трогайте.
5. Нажмите **Save**. Email/Password появится в списке со статусом Enabled.

На вкладке **Users** будут появляться зарегистрированные пользователи. Там же есть кнопка **Add user** — так можно создать тестового пользователя вручную, без кода.

8. Первый пользователь из приложения

Добавим в приложение кнопку, которая регистрирует тестового пользователя. Это проверка, что связка «приложение — Firebase» работает.

```
import 'package:firebase_auth/firebase_auth.dart';

Future<void> _register() async {
 try {
  final cred = await FirebaseAuth.instance
     .createUserWithEmailAndPassword(
    email: 'test1@test.com',
    password: '123456',
  );
  print('Создан пользователь: ${cred.user?.uid}');
 } on FirebaseAuthException catch (e) {
  print('Ошибка: ${e.code}');
 }
}

// где-нибудь на экране:
ElevatedButton(
 onPressed: _register,
 child: const Text('Зарегистрироваться'),
)
```

Нажмите кнопку, затем обновите вкладку Users в консоли. Там появится **test1@test.com** и его User UID — уникальный идентификатор пользователя. Именно по uid мы потом будем привязывать к пользователю его избранное.

Если нажать кнопку второй раз, в консоли отладки будет **Ошибка: email-already-in-use**. Это нормально: такой пользователь уже есть. Вход и выход выглядят так:

```
// вход
await FirebaseAuth.instance.signInWithEmailAndPassword(
  email: 'test1@test.com',
  password: '123456',
);

// кто сейчас вошёл (null — никто)
final user = FirebaseAuth.instance.currentUser;

// выход
await FirebaseAuth.instance.signOut();
```

Таблица 2. Частые коды ошибок входа

Код ошибки	Что значит	Что сделать
email-already-in-use	Пользователь с такой почтой уже есть	Войти через signIn или взять другую тестовую почту
weak-password	Пароль короче 6 символов	Придумать пароль подлиннее
invalid-email	Почта записана с ошибкой	Проверить формат: имя@домен
invalid-credential	Неверная почта или пароль при входе	Проверить данные или зарегистрироваться
operation-not-allowed	Способ входа не включён	Включить Email/Password (раздел 7)
network-request-failed	Нет интернета	Проверить сеть, на эмуляторе — его подключение

Источник: составлено автором.

9. Простой экран входа

Экран входа можно сделать на том, что вы уже знаете: Future, setState и навигация. Приложение при запуске смотрит, вошёл ли пользователь, после входа само переходит на главный экран, а после выхода — обратно на экран входа.

При запуске: кто уже вошёл?

Firebase помнит вход между запусками. **currentUser** — пользователь, который сейчас вошёл, или **null**.

```
class MyApp extends StatelessWidget {
 const MyApp({super.key});


 @override
 Widget build(BuildContext context) {
  final user = FirebaseAuth.instance.currentUser;
  return MaterialApp(
   home: user == null ? const LoginScreen() : const HomeScreen(),
  );
 }
}
```

Войти и перейти на главный экран

В **LoginScreen** два поля с контроллерами (**_email**, **_password**) и поле **_error** для текста ошибки. Метод входа:

```
Future<void> _signIn() async {
 try {
  await FirebaseAuth.instance.signInWithEmailAndPassword(
   email: _email.text.trim(),
   password: _password.text,
  );
  if (!mounted) return;
  Navigator.pushReplacement(
   context,
   MaterialPageRoute(builder: (_) => const HomeScreen()),
  );
 } on FirebaseAuthException catch (e) {
  setState(() => _error = e.code);
 }
}
```

pushReplacement убирает экран входа из стопки, чтобы кнопка «назад» на него не возвращала. Регистрация выглядит так же, только вместо **signInWithEmailAndPassword** вызывается **createUserWithEmailAndPassword**.

Выйти и вернуться на экран входа

```
IconButton(
 icon: const Icon(Icons.logout),
 onPressed: () async {
  await FirebaseAuth.instance.signOut();
  if (!context.mounted) return;
  Navigator.pushReplacement(
   context,
   MaterialPageRoute(builder: (_) => const LoginScreen()),
  );
 },
)
```

У простого варианта есть ограничения: переходы приходится писать самим, экран не узнает о входе или выходе, случившемся не по кнопке, а в вебе при запуске **currentUser** ещё может быть **null**. На следующей лекции разберём, как это исправить через Stream: экран будет сам узнавать о входе и выходе.

10. Что коммитить в Git

Ключи в **firebase_options.dart**, **google-services.json** и **GoogleService-Info.plist** — не секрет. Они зашиты в каждое опубликованное приложение, их можно достать из любого APK или сайта. Данные защищают не ключи, а правила доступа к базе и вход пользователей.

- В учебном репозитории коммитим всё, включая **firebase.json** и **.firebaserc**. Так проект собирается сразу после клонирования. GitHub

может прислать письмо «Google API key exposed» — это ожидаемо, алерт можно закрыть.

- **В личном публичном проекте** три конфига можно убрать в **.gitignore**, а в README написать команду **flutterfire configure --project=...**: файлы создаются ею заново.
- **Настоящий секрет** — JSON сервисного аккаунта (***-firebase-adminsdk-*.json**). Его никогда не коммитим и не показываем на экране. В нашем курсе он не нужен.

Эти строки стоит добавить в **.gitignore** всегда:

```
# кэш Firebase Hosting
.firebase/
# логи CLI
firebase-debug.log
# ключи сервисного аккаунта
*-firebase-adminsdk-*.json
```

11. Если что-то пошло не так

Таблица 4. Частые проблемы и решения

Проблема	Причина	Решение
npm ERR! EACCES (macOS)	У npm нет прав на системную папку	**sudo npm install -g firebase-tools**
npm WARN EBADENGINE	Старый Node.js	Обновить Node до 22 или новее
Windows: «выполнение сценариев отключено»	PowerShell запрещает скрипты	**Set-ExecutionPolicy -Scope CurrentUser RemoteSigned**
flutterfire: command not found	Папка с утилитами Dart не в PATH	Раздел 4, затем перезапустить терминал
FirebaseProjectNotFoundException или проекта нет в **firebase projects:list**	CLI вошёл другим Google-аккаунтом	**firebase login:add**, затем **firebase login:use почта**
Ошибка про **XCLocalSwiftPackageReference**	macOS: старая библиотека xcodeproj у системного Ruby	Только **--platforms=android,web** или Ruby через Homebrew (раздел 5)
Ошибка про **minSdkVersion** при сборке Android	Пакетам Firebase нужна более новая версия Android	В **android/app/build.gradle.kts** (или **build.gradle**) поставить **minSdk = 23**

Проблема	Причина	Решение
rsync ... SourcePackages ... failed при **flutter pub get** (macOS)	После **flutter clean** нет папки **build/ios**	**mkdir -p build/ios/SourcePackages** и снова **flutter pub get**
Binding has not yet been initialized	Нет **ensureInitialized()** перед **await**	Первой строкой в **main**: **WidgetsFlutterBinding.ensureInitialized( )**
No Firebase App '[DEFAULT]' has been created	Firebase не запущен до использования	Проверить **main()** из раздела 6: есть ли **await Firebase.initializeApp**
network-request-failed на эмуляторе	У эмулятора нет интернета	Cold Boot эмулятора в Device Manager, проверить сайт в его Chrome, выключить VPN
operation-not-allowed	Вход по email не включён	Раздел 7
Жёлтое **WARNING** про Kotlin Gradle Plugin	Плагины Firebase ещё не перешли на новый способ	Не ошибка, можно игнорировать
Вместо приложения заглушка «Firebase Hosting Setup Complete»	**public** вместо **build/web** или перезаписан **index.html**	Раздел 11: проверить **firebase.json**, пересобрать и задеплоить

Источник: составлено автором.

Не получается за 15–20 минут — напишите в чат курса: что делали, какую команду вводили и скриншот ошибки целиком.

12. Шпаргалка

Таблица 5. Команды из этого гайда

Команда	Что делает	Когда
npm install -g firebase-tools	Ставит Firebase CLI	Один раз на компьютер
firebase login	Вход в Google-аккаунт	Один раз на компьютер
firebase projects:list	Список ваших проектов	Для проверки
firebase login:add / **login:use**	Добавить и выбрать другой аккаунт	Если проекта нет в списке
dart pub global activate flutterfire_cli	Ставит FlutterFire CLI	Один раз на компьютер
flutterfire configure --project=... --platforms=android,web	Настраивает Flutter-проект	Один раз на проект и при новых платформах
flutter pub add firebase_core firebase_auth	Добавляет пакеты	Один раз на проект
flutter build web	Собирает веб-версию	Перед каждой публикацией
firebase init hosting	Настраивает Hosting	Один раз на проект
firebase deploy --only hosting	Публикует сайт	После каждой сборки

Источник: составлено автором.

Источники

[1] Add Firebase to your Flutter app. — URL: https://firebase.google.com/docs/flutter/setup

[2] Firebase CLI reference. — URL: https://firebase.google.com/docs/cli

[3] Authenticate with Firebase using Password-Based Accounts on Flutter. — URL: https://firebase.google.com/docs/auth/flutter/password-auth

[4] Get started with Firebase Hosting. — URL: https://firebase.google.com/docs/hosting/quickstart